

Sistema de Gestión de Base de Datos

Simulación de Almacenamiento Secundario

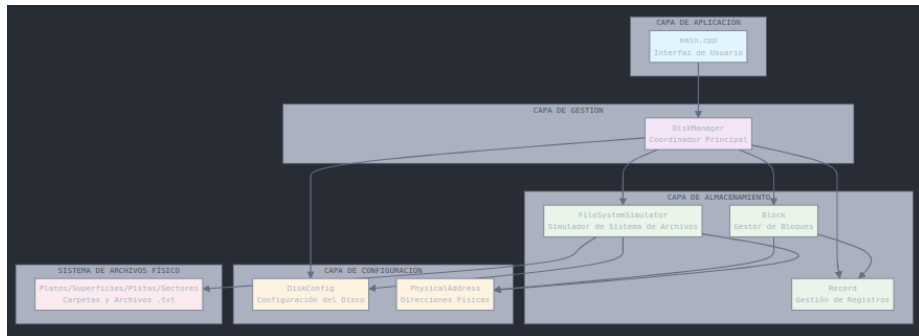
Wilson Ramos Pacco

Universidad Nacional de San Agustín de Arequipa

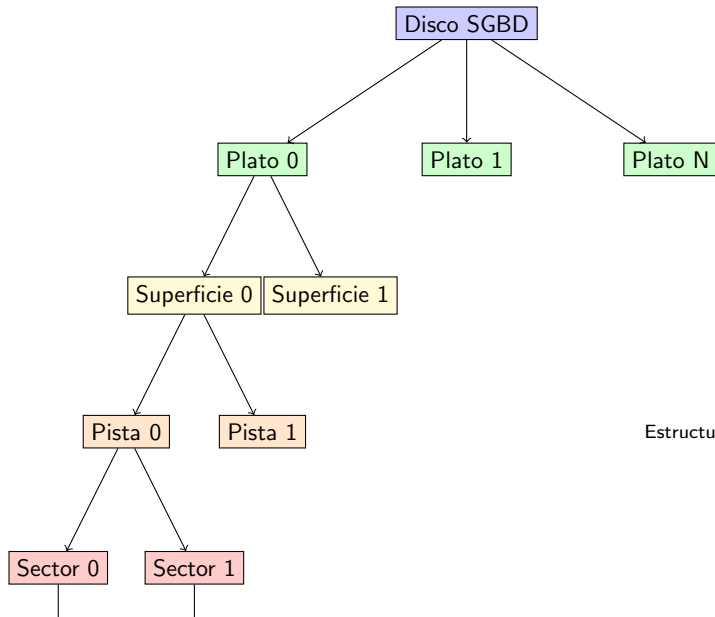
10 de junio de 2025

- 1 Arquitectura del Sistema
- 2 Diagrama de Clases
- 3 Implementación Clave
- 4 Trabajo Futuro

Diagrama de Arquitectura General

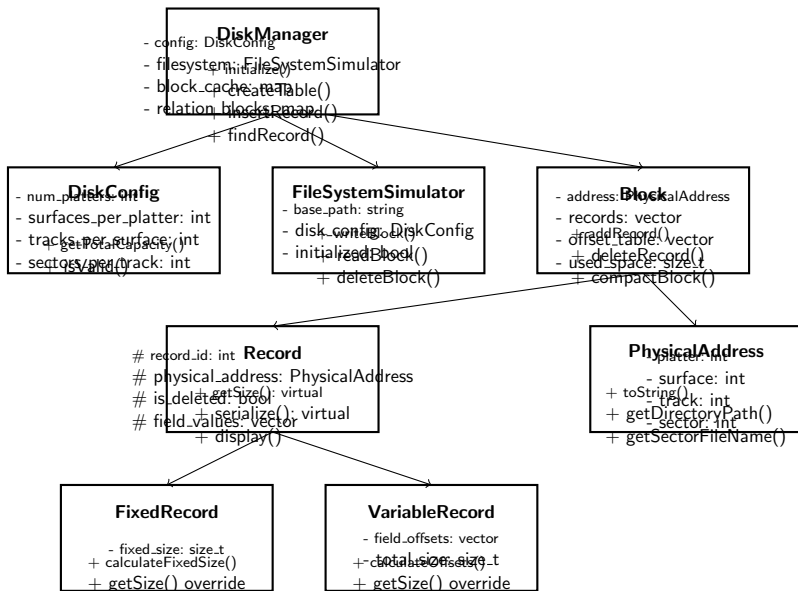


Estructura Jerárquica del Disco Simulado



Estructura enMemoria

Diagrama de Clases Principal



Listing 1: PhysicalAddress.h - Representación de direcciones

```
class PhysicalAddress {
private:
    int platter;    // N mero de plato (0-n)
    int surface;    // N mero de superficie (0-1)
    int track;      // N mero de pista/cilindro (0-n)
    int sector;     // N mero de sector (0-n)

public:
    PhysicalAddress(int p, int s, int t, int sec)
        : platter(p), surface(s), track(t), sector(sec) {}

    std::string toString() const {
        std::ostringstream oss;
        oss << "P" << platter << "_S" << surface
            << "_T" << track << "_SEC" << sector;
        return oss.str();
    }

    std::string getDirectoryPath() const {
        std::ostringstream oss;
        oss << "platter_" << platter << "/surface_" << surface
            << "/track_" << track;
        return oss.str();
    }
};
```

Configuración del Disco

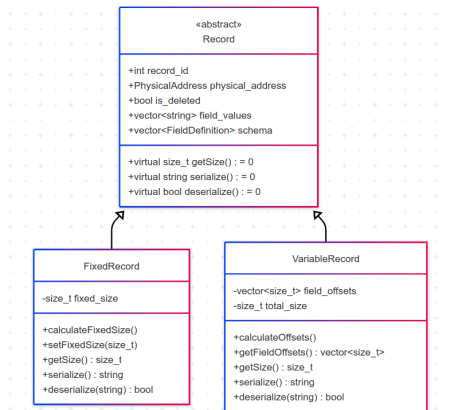
```
class DiskConfig {
private:
    int num_platters;
    int surfaces_per_platter;
    int tracks_per_surface;
    int sectors_per_track;
    int bytes_per_sector;
    // Par metros de rendimiento (basados en
    // Megatron 747)
    double seek_time_ms; // 6.46 ms promedio
    double rotational_latency_ms; // 4.17 ms
    // promedio
    double transfer_time_ms; // 0.13 ms por
    // sector

public:
    long long getTotalCapacity() const {
        return static_cast<long long>(num_platters)
            * surfaces_per_platter *
            tracks_per_surface *
            sectors_per_track * bytes_per_sector
            ;
    }
    std::string getFormattedCapacity() const {
        long long bytes = getTotalCapacity();
        if (bytes >= (1LL << 40)) {
            return std::to_string(bytes / (1LL <<
                40)) + " TB";
        }
        // ... m s conversiones de unidades
    }
};
```

```
#include <iostream>
#include "DiskConfig.h"
int main() {
    DiskConfig DiskMEGATRON;
    std::cout << "Capacidad: " << DiskMEGATRON.getFormattedCapacity() << "\n";
    std::cout << "Sectores: " << DiskMEGATRON.getTotalSectors() << std::endl;
```

```
• Capacidad: 512 GB
• developer@developer-ThinkPad-Yoga-460:~/UNSA-2025A/Base de Datos II/sgbd-proyecto/inc
Capacidad: 512 GB
Sectores: 134217728
```

Gestión de Registros



Listing 2: Record.h - Implementación de registros fijos

```
class FixedRecord : public Record {
private:
    size_t fixed_size;

public:
    void calculateFixedSize() {
        fixed_size = sizeof(int) + sizeof(bool); // ID + deleted flag

        for (const auto& field : schema) {
            switch (field.type) {
                case FieldType::INTEGER:
                    fixed_size += sizeof(int);
                    break;
                case FieldType::FLOAT:
                    fixed_size += sizeof(float);
                    break;
                case FieldType::STRING:
                    fixed_size += field.max_length; // Tamaño fijo
                    break;
                case FieldType::DATE:
                    fixed_size += 12; // "YYYY-MM-DD\0"
                    break;
            }
        }

        // Alineación múltiplo de 4 bytes
        fixed_size = (fixed_size + 3) & ~3;
    }
};
```

Listing 3: Record.h - Implementación de registros variables

```
class VariableRecord : public Record {
private:
    std::vector<size_t> field_offsets; // Offsets de cada campo
    size_t total_size;

public:
    void calculateOffsets() {
        field_offsets.clear();
        total_size = sizeof(int) + sizeof(bool) + sizeof(size_t);

        // Espacio para offsets en el header
        total_size += schema.size() * sizeof(size_t);

        // Calcular offsets de cada campo
        for (size_t i = 0; i < field_values.size(); ++i) {
            field_offsets.push_back(total_size);

            switch (schema[i].type) {
                case FieldType::STRING:
                    // Longitud variable + terminador nulo
                    total_size += field_values[i].length() + 1;
                    break;
                // ... otros tipos
            }
        }
    };
};
```

```
std::shared_ptr<Record> findRecord(int record_id) const {  
    for (const auto& record : records) { // Recorrido secuencial  
        if (record->getId() == record_id && !record->isDeleted()) {  
            return record;  
        }  
    }  
    return nullptr;  
}
```

R. Fijo

$size_t record_position = record_number * fixed_size;$

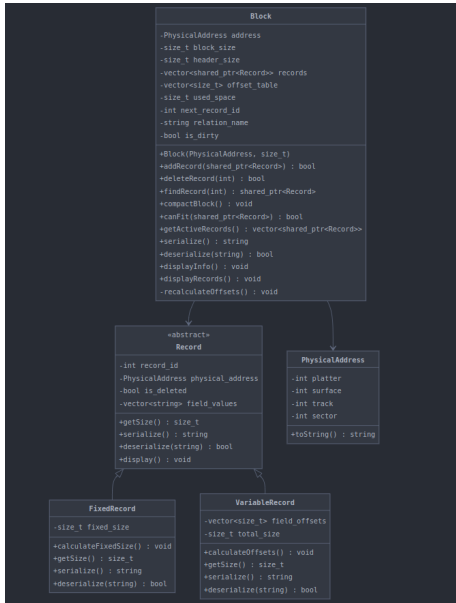
R.variable

$std :: vector < sizet >$

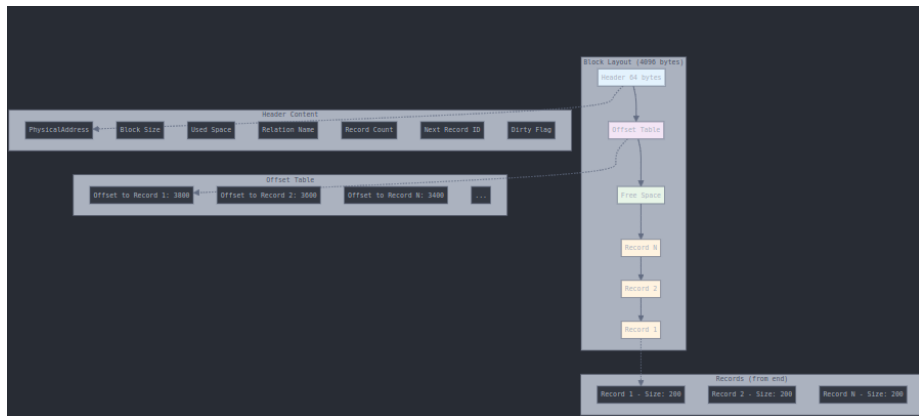
$fieldoffsets; // D\acute{o}nde empieza cada campo // Para acceder al campo N :$

$sizet fieldstart = fieldoffsets[N]; // Leer desde esa posici\acute{o}n$

Gestión de Bloques



Estructura interna del Bloque



Compactacion

```
void compactBlock() {  
    // Filtrar registros no eliminados  
    auto new_end = std::remove_if(records.begin(), records.end(),  
        [](const std::shared_ptr<Record>& record) {  
            return record->isDeleted();  
        });  
  
    records.erase(new_end, records.end());  
  
    // Recalcular offsets y espacio usado  
    recalculateOffsets();  
    markDirty();  
}
```

paso1 std::remove; *fReorganizacion*

paso2 : erase() — *Eliminacion Fisica(desdenew_e nhastaelfinal)*

Paso3 : recalculateOffsets() — *Reconstruir Tabla*

Simulacion Compactacion

DEMOSTRACIÓN COMPLETA DEL PROCESO DE COMPACTACIÓN

=== DATOS DE PRUEBA INICIALES ===

Estado inicial:

Índice:	[0]	[1]	[2]	[3]	[4]	[5]	[6]
Record:	R1	R2	R3	R4	R5	R6	R7
Estado:	ACT	DEL	ACT	DEL	ACT	DEL	ACT
Contenido:	Dat	Dat	Dat	Dat	Dat	Dat	Dat

=== SIMULACIÓN MANUAL DE std::remove_if ===

Proceso paso a paso:

Lambda: [](const Record& r) { return r.isDeleted(); }

Objetivo: Mover elementos NO eliminados al frente

--- Iteración 1 ---

Leyendo posición 0: R1 (Estado: ACTIVO)

- NO eliminado: copiar a posición 0
- Sin movimiento necesario (misma posición)
- write_pos avanza a: 1
- read_pos avanza a: 1

Estado actual del vector:

Válidos: [0..1), Basura: [1..7)

R1 | R2 R3 R4 R5 R6 R7

--- Iteración 2 ---

Leyendo posición 1: R2 (Estado: ELIMINADO)

- ELIMINADO: saltar (no copiar)
- read_pos avanza a: 2

Estado actual del vector:

Válidos: [0..1), Basura: [1..7)

R1 | R2 R3 R4 R5 R6 R7

--- Iteración 3 ---

--- Iteración 4 ---

Leyendo posición 3: R4 (Estado: ELIMINADO)

- ELIMINADO: saltar (no copiar)
- read_pos avanza a: 4

Estado actual del vector:

Válidos: [0..2), Basura: [2..7)

R1 R3 | R3 R4 R5 R6 R7

--- Iteración 5 ---

Leyendo posición 4: R5 (Estado: ACTIVO)

- NO eliminado: copiar a posición 2
- Movimiento: R5 desde [4] hacia [2]
- write_pos avanza a: 3
- read_pos avanza a: 5

Estado actual del vector:

Válidos: [0..3), Basura: [3..7)

R1 R3 R5 | R4 R5 R6 R7

--- Iteración 6 ---

Leyendo posición 5: R6 (Estado: ELIMINADO)

- ELIMINADO: saltar (no copiar)
- read_pos avanza a: 6

Estado actual del vector:

Válidos: [0..3), Basura: [3..7)

R1 R3 R5 | R4 R5 R6 R7

--- Iteración 7 ---

Leyendo posición 6: R7 (Estado: ACTIVO)

- NO eliminado: copiar a posición 3
- Movimiento: R7 desde [6] hacia [3]
- write_pos avanza a: 4

Listing 4: FileSystemSimulator.h - Escribir bloque a disco

```
class FileSystemSimulator {
public:
    bool writeBlock(const PhysicalAddress& address, const Block& block) {
        if (!isValidAddress(address)) {
            return false;
        }

        std::string file_path = getFullPath(address);

        // Crear directorios si no existen
        fs::create_directories(fs::path(file_path).parent_path());

        std::ofstream file(file_path);
        if (!file.is_open()) {
            return false;
        }

        // Escribir metadatos del sector
        file << "# Sector: " << address.toString() << std::endl;
        file << "# Fecha: " << getCurrentTimestamp() << std::endl;
        file << "# Relaci n: " << block.getRelationName() << std::endl;
        file << "# Registros: " << block.getRecordCount() << std::endl;
        file << "# Ocupaci n: " << block.getOccupancyPercentage()
            << "%" << std::endl;
        file << "# -----" << std::endl;

        // Escribir contenido serializado del bloque
        file << block.serialize();

        return true;
    }
};
```


Simulación de Tiempos de Acceso

Listing 5: DiskManager.h - Cálculo de tiempos de acceso

```
private:
    double simulateAccessTime(const PhysicalAddress& addr) {
        static std::random_device rd;
        static std::mt19937 gen(rd());

        // Seek time (variable seg n distancia)
        std::uniform_real_distribution<> seek_dist(
            0.0, config.getSeekTime() * 2);
        double seek_time = seek_dist(gen);

        // Rotational latency (0 a m ximo)
        std::uniform_real_distribution<> rot_dist(
            0.0, config.getRotationalLatency() * 2);
        double rot_latency = rot_dist(gen);

        // Transfer time (fijo)
        double transfer_time = config.getTransferTime();

        return seek_time + rot_latency + transfer_time;
    }

public:
    bool insertRecord(const std::string& table_name,
                     const std::vector<std::string>& values) {
        // ... crear registro y encontrar bloque ...

        if (block->addRecord(record)) {
            // Simular tiempo de escritura
            double access_time = simulateAccessTime(block->getAddress());
            total_access_time += access_time;
            total_writes++;

            std::cout << "Registro insertado (Tiempo: "
                      << access_time << " ms)" << std::endl;

            return true;
        }
    }
}
```

Contenido de un Sector

```
test_disk_records / platter_0 / surface_0 / track_0 / 2 sector_0.txt
# Sector: P0_S0_T0_SEC0
# Fecha: 2025-06-10 10:04:47
# Relación: empleados_fijos
# Registros: 5
# Ocupación: 9.86328%
# =====
BLOCK_HEADER|P0_S0_T0_SEC0|4096|404|empleados_fijos|5
OFFSET_TABLE|64,132,200,268,336
RECORD|FIXED|1|0|P0_S0_T0_SEC0|1,Juan Perez,30,75000.50,2020-01-15
RECORD|FIXED|2|0|P0_S0_T0_SEC0|2,Maria Garcia,28,68000.00,2019-03-22
RECORD|FIXED|3|0|P0_S0_T0_SEC0|3,Carlos Rodriguez,35,82000.75,2018-07-10
RECORD|FIXED|4|0|P0_S0_T0_SEC0|4,Ana Martinez,32,71500.25,2021-02-18
RECORD|FIXED|5|0|P0_S0_T0_SEC0|5,Luis Gonzalez,29,63000.00,2020-11-05
```

- **Header:** Metadatos del sector con timestamp
- **Block Header:** Información estructural del bloque
- **Offset Table:** Posiciones de cada registro
- **Records:** Datos serializados con tipo y estado

Simulacion espacio insuficiente

```
=== SIMULACION: ESPACIO INSUFICIENTE ===  
Tabla para simulacion: Titanic  
  
ESCENARIO SIMULADO:  
- Sector actual: Plato_0/Superficie_0/Pista_2/Sector_15  
- Tamaño sector: 4096 bytes  
- Espacio usado: 3900 bytes  
- Espacio libre: 196 bytes  
- Registro nuevo: 512 bytes  
  
RESULTADO: Espacio insuficiente (deficit: 316 bytes)  
  
SOLUCION:  
1. Buscar proximo sector disponible  
2. Sector_18 encontrado con 2048 bytes libres  
3. Registro asignado al nuevo sector  
4. Insercion completada exitosamente
```

Operaciones CRUD - Casos de Uso

Inserción de Registros

- 1 Crear tabla con esquema
- 2 Insertar registros individuales
- 3 Cargar lotes desde CSV
- 4 Verificar ubicación física

Eliminación de Registros

- 1 Eliminación lógica (tombstone)
- 2 Compactación de bloques
- 3 Liberación de espacio
- 4 Actualización de metadatos

Búsqueda de Registros

- 1 Búsqueda por ID
- 2 Simulación de tiempos de acceso
- 3 Mostrar dirección física
- 4 Estadísticas de rendimiento

Gestión de Espacio

- 1 Cálculo de ocupación
- 2 Asignación de nuevos bloques
- 3 Fragmentación del disco
- 4 Optimización de almacenamiento

Extensiones Planificadas

1 Índices B+ Tree

- Búsquedas eficientes
- Ordenamiento automático
- Rangos de consulta

2 Gestión de Transacciones

- Control de concurrencia
- Logging de operaciones
- Rollback y commit

3 Optimización de Consultas

- Parser SQL básico
- Planes de ejecución
- Estadísticas de costo

Mejoras Técnicas

1 Gestión de Buffer Pool

- Cache inteligente de bloques
- Algoritmos LRU/Clock
- Dirty page management

2 Recuperación ante Fallos

- Write-ahead logging
- Checkpointing
- Recovery algorithms

3 Interfaz Gráfica

- Visualización de estructura
- Monitor de rendimiento
- Herramientas de depuración

¡Gracias!

Preguntas y Discusión

Repositorio: <https://github.com/WilsonRamos/sgbd-proyecto>
Demos: [examples/](#)
Tests: [tests/](#)

Sistema de Gestión de Base de Datos Físico